

README of Project for LLM 'Scientia'

Inhaltsverzeichnis

1. Overview
2. *scientia-base* - **Leitfaden zur Weiterentwicklung**
3. Netzwerkpipeline
4. Trainingsdaten
5. Befehle
6. Starten unter UNIX von 0

=> **Schauen Sie auch Planungsdokument**

Wenn Sie der Serveradmin sind und schnell Anfangen wollen empfehle ich Ihnen mit 6. Starten unter UNIX von 0 zu beginnen und sich dannach 3. Netzwerkpipeline, 5. Befehle und 2. *scientia-base* - **Leitfaden zur Weiterentwicklung** durchzugehen. Wenn Sie einfach nur wissen wollen wie man die Trainingsdaten extrahiert nutzen Sie 4. Trainingsdaten

1. Overview

- Author: *Pydalo / Peter Helmut Wohlfarth*
- Date (begin): 6-6-2026
- Use: **Chatbot** for students of *HTWK Leipzig*
- Language: German (de)
- Operating System: Windows OS (v10 + 11)

2. *scientia-base* - Leitfaden zur Weiterentwicklung

Struktureller Dateiaufbau

Folgendes zeigt den Dateiaufbau des Repositories:

```
.
├── .git
├── .idea
├── data
│   ├── veclib
│   └── md
├── docs
│   ├── ChatbotPlanung.odt
│   ├── ChatbotPlanung.pdf
│   ├── PlanPraktikum.odt
│   ├── PlanPraktikum.pdf
│   └── USAGE.md
├── README.md
├── scientia
│   ├── .idea
│   │   ├── .gitignore
│   │   ├── inspectionProfiles
│   │   │   └── profiles_settings.xml
│   │   ├── misc.xml
│   │   ├── modules.xml
│   │   ├── runConfigurations
│   │   │   └── Backend.xml
│   │   ├── Scientia.iml
│   │   └── vcs.xml
│   ├── .venv
│   │   └── .gitignore
│   ├── data
│   │   ├── all.jsonl
│   │   ├── train.jsonl
│   │   ├── val.jsonl
│   │   └── veclib
│   │       ├── md
│   │       ├── raw
│   │       ├── text_chunks.pkl
│   │       └── vektorbase.index
│   ├── requirements.txt
│   ├── run
│   │   ├── backend.py
│   │   └── run.py
│   ├── test.py
│   ├── training
│   └── download.py
```

```

|   |   └─ finetune.py
|   |   └─ split.py
|   └─ veclib
|       └─ convmarkdown.py
|       └─ convmarkdownsingle.py
|       └─ genvectorlib.py
└─ server
    └─ config.js
    └─ node_modules
    └─ package-lock.json
    └─ package.json
    └─ public
        └─ animation.js
        └─ assets
            └─ background.png
            └─ faviconv2.png
        └─ index.html
        └─ main.js
        └─ style.css
    └─ server.js
    └─ start.bat

```

Projekt Fraktionen Überblick

Bevor Sie irgendetwas tun, überprüfen Sie, ob Python, Nodejs und npm installiert sind. Die Befehle dafür finden Sie in der mittleren Spalte, falls dabei aber nicht die Version in der linken Spalte zu sehen sind, nutzen Sie den Befehl aus der Rechten Spalte um die Abhängigkeit **GLOBAL** zu installieren, falls Sie windows nutzen, müssen Sie die letzte Spalte nutzen, um die Abhängigkeit manuell über den Browser zu installieren:

Software	Version	Prüfbefehl	Installation (Linux/macOS)	Installation (Windows)
Node.js	v24.11.0	node -v	curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh bash && source ~/.bashrc && nvm install 24.11.0 && nvm use 24.11.0	https://nodejs.org
npm	v11.16.0	npm -v	npm install -g npm@11.16.0	Wird automatisch mit Node.js installiert.
Python	v3.13.14	python --version	curl https://pyenv.run bash && exec \$SHELL && pyenv install 3.13.14 && pyenv global 3.13.14	https://www.python.org/downloads/

Das gesamte Projekt lässt sich in zwei Fraktionen unterteilen:

./server - Serverapplikationen

Hier liegt der NodeJS-Server und die Nutzerwebsite und alle dazu gehörigen Applikationen. Folgende Unterordner bzw Unterdateien sind in dem Ordner zu finden und wichtig zu erwähnen:

- **/public** - Hier liegt die Website für den Nutzer
- **server.js** - Hier liegt die Servapplikation für den Nodejs-Server. Es ist wichtig, dass Sie vor dem ersten Start alle Abhängigkeiten installieren, was mit einem Zum Starten des Servers geben sie aus dem Root-Directory (Hauptordner des Projektes) folgenden Befehl ein:

```
cd ./server | node server.js
```

- **config.js** - Hier kann der Nodejs-Server konfiguriert werden, folgende Optionen stehen zur Auswahl:
 - **config.port** - Der Port auf dem der Nodejs-Server läuft (Integer)
 - **config.host** - Der Host unter dem der Nodejs-Server aufrufbar ist (String)
 - **config.windows** - Ist **true** wenn der Nodejs-Server auf WindowsOS läuft, jeder andere Wert bedeutet, dass der Server auf Linux oder macOS (ist vom Kernel dasselbe) läuft (Boolean)
 - **config.debuglevel** - Teilt den Nodejs-Server mit wie viel er an Debug-Nachrichten ausgeben soll. Dabei bedeuten folgende Werte folgendes für den Nodejs-Server:
 - **-1** - Der Nodejs Server gibt **keine** Debug-Nachricht aus (außer Fehlermeldungen auf dem Server selbst)
 - **0** - Der Nodejs Server gibt **jede** Debug-Nachricht aus
 - **1** - Der Nodejs Server gibt jede Debug-Nachricht aus, bis auf **Netzwerkkommunikations-Debug-Nachrichten** zwischen Client und Nodejs Server

- **package.json** - Enthält **alle benötigten Abhängigkeiten bzw Bibliotheken** zum Nodejs-Server-Betrieb im JSON-Format. Beim erstmaligen Start des Nodejs-Servers werden alle Abhängigkeiten **automatisch installiert** (solange die richtige Nodejs-Version `v24.11.0` benutzt wird).

./scientia - Der eigentliche Chatbot

Hier liegen alle Bibliotheken und ein gesamtes Python-Virtual-Environment. Ziel hierbei ist die Bereitstellungen des Chatbots, also das Laden, Ausführen, Trainieren und Bilden einer Datenbasis des Chatbots. Damit der Nutzer mit den Chatbot interagieren kann gibt es in der `backend.py` eine API (zum Austausch). Folgende Unterordner bzw Unterdateien sind in dem Ordner zu finden und wichtig zu erwähnen:

- **./venv** - Enthält die Python-Virtual-Environment also die Runtime für das AI-Python-Backend. Der Ordner **muss mittels eines Befehls von Python erstellt werden**, da dieser Ordner betriebssystemspezifische Dateien enthält. Hiermit kann ein Virtual-Environment erstellt werden:

```
python -m venv ./venv
```

Beachten Sie, dass Python 3.11 erforderlich ist, da das AI-Backend dafür ausgelegt ist und Sie sich im richtigen Ordner befinden, was für alle folgenden, `./scientia`-betreffenden Befehlen sehr wichtig ist, dafür navigieren Sie in den Ordner `./scientia`, was sie aus dem Root-Directory (Der Hauptordner des Projektes) mit folgenden Befehl tun:

```
cd ./scientia
```

. Um mit `pip` das Projekt konfigurieren wollen, müssen Sie folgenden Befehl eintippen, um immer im Virtual-Environment zu sein: für Linux:

```
source ./venv/bin/activate
```

und für Windows nur:

```
./venv/Scripts/activate
```

- **requirements.txt** - Diese Datei enthält (ähnlich wie die package.json) alle Abhängigkeiten, die für das AI-Backend erforderlich sind. Um diese Abhängigkeiten für das spezifische Betriebssystem in das `./venv` herunterzuladen muss folgender Befehl eingetippt werden, beachten Sie dieses mal auch, dass pip3.11 installiert sein muss:

```
pip install -r ./requirements.txt
```

und Sie sich im Root-Directory befinden müssen. Falls Sie während der Entwicklung neue Bibliotheken mit:

```
pip install package-name
```

installieren, müssen Sie die `requirements.txt` updaten, was Sie mit folgenden Befehl tun:

```
pip freeze > ./requirements.txt
```

- **run/backend.py** - Diese Datei enthält die Applikation für den AI-Backend-Server. Beim starten (was automatisch durch den Nodejs-Server passiert) wird ein flask-Server gestartet und das/die KI-Modell(e) geladen.
- **run/config.py** - Hier kann der AI-Backend-Server konfiguriert werden. Folgende Konfigurationsmöglichkeiten stehen zur Auswahl:
 - `config.LLM_PATH` - Der Pfad zum LLM-Modell (large language model) (String)
 - `config.EMB_PATH` - Der Pfad zum Embedding-Modell (String)
 - `config.INDEX_FILE` - Der Pfad zu der Index-Datei der Vektorbibliothek (String)
 - `config.CHUNKS_FILE` - Der Pfad zu der Pfad zu den rohen Chunks der Vektorbibliothek
 - `config.HOST` - Der Host, auf dem der AI-Backend-Server läuft und für Nodejs-Server zugänglich ist (String)
 - `config.PORT` - Der Port auf dem der AI-Backend-Server läuft und für Nodejs-Server zugänglich ist (Integer)
 - `config.THREADS` - Die Anzahl an Threads, die der AI-Backend-Server zum parallelen Verarbeiten zur Verfügung hat (Integer)
 - `config.SYSTEM_PROMPT` - Der Systemprompt für die Nutzeranfragen für das LLM (String)
 - `config.EXTENDED_CONTEXT(context_text, user_query)` - Das Format für die Zusatzinformationen aus der Vektorbibliothek und der Nutzerfrage. `context_text` ist die Zusatzinformation aus der Vektorbibliothek und `user_query` ist die Nutzerfrage (Function)
- **run/run.py** - Eine Testdatei um ein KI-Modell zu testen, das KI-Modell liegt unter dem Pfad unter `config.LLM_PATH` in der `run/config.py`.
- **training/download.py** - Diese Datei lädt ein KI-Modell von Huggingface herunter. Dabei können zwei Argumente an die Datei übergeben werden, wie hierfolgt dargestellt:

```
python ./training/download.py "huggingface_modell_pfad" "lokaler_pfad_relativ_zum_current_dir"
```

Falls keine Parameter angegeben werden, werden folgende zwei Modelle installiert:

- `Qwen/Qwen3-4B-Instruct-2507` unter den Dateipfad (rel. zum Root): `../models/Qwen/Qwen3-4B-Instruct-2507` - das Hauptmodell
 - `intfloat/multilingual-e5-small` unter den Dateipfad (rel. zum Root): `../models/intfloat/multilingual-e5-small` - Das Vektorbibliotheken Modell

- **training/split.py** - Diese Datei teilt einen Trainingsdatensatz in einer `all.json` in zwei weitere Dateien auf n hmlich die `train.json` und `val.json`, die f r das Training essentiell sind. Unter der Bedingung, dass man sich im Virtual-Enviroment befindet (siehe oben unter `./venv`) kann die Datei mit folgenden Befehl genutzt werden:

```
python train/split.py "pfad_zur_all.json" "pfad_zur_train.json" "pfad_zur_val.json"
```

- **veclib/convmarkdown.py** - Diese Datei  bersetzt alle PDF-s aus `./data/veclib/raw` mit einem *nicht so guten* KI-Modell zu Markdown nach `./data/veclib/md`. Ich empfehle daher die im n chsten Abschnitt 3. Trainingsdaten Methode zu nutzen.
- **veclib/convmarkdownsingle.py** - Diese Datei ist nahezu identisch zu `vec/convmarkdownsingl`
`e.py` blo , dass diese Datei eine einzelne Datei  bersetzt. Nach dem starten dieser Datei mit dem folgenden Befehl, werden Sie nach dem Dateipfad gefragt:

```
> python ./veclib/convmarkdownsingle.py
Pfad zur Datei / Path to file: "test/test/vorlesung.pdf" # <-- Ihr Dateipfad hier
Starte Konvertierung...
Verarbeite: vorlesung.pdf
Erfolgreich konvertiert: vorlesung.pdf (21 Bilder extrahiert)
Fertig!
```

- **veclib/genvectorlib.py** - Diese Datei generiert aus allen Markdowndateien unter `./data/veclib/md` eine Vektorbibliothek unter `./data/veclib/vektorbases.index` und `./data/veclib/text_chunks.pkl`. Dabei wird ein KI-Modell verwendet, dass unter den Pfad in `run/config.EMB_PATH`.

3. Netzwerkpipeline

Wie oben zu sehen gibt es zwei Server. Um nun das Zusammenspiel beider Server zu verstehen gehen wir einmal die gesamte Pipeline von der Nutzerfrage bis zum Ergebniss durch:

1. **Client** - Der Nutzer tippt eine *Frage* in ein Input-Feld auf der Website ein und sendet ab.
2. **Client** - Der Client h ngt die *Frage* hinten an die Chathistory zu einen *Prompt* an.
3. **Client** → **NodeJS-Server** - Der Client schickt den *Prompt* zum NodeJS-Server und baut eine Token-Stream-Connection  ber das Internet mit dem Client auf
4. **NodeJS-Server** → **AI-Python-Backend-Server** - Der NodeJS-Server nimmt dem *Prompt* entgegen und leitet ihn direkt zum AI-Python-Backend-Server weiter. Beide Server laufen auf dem selben Hardware-Server
5. **AI-Python-Backend-Server** - Der *Prompt* vom Client wird eventuell gek rzt und der Systemprompt angehangen.
6. **AI-Python-Backend-Server** - Das Vektorbibliotheken Modell wird gestartet und die Vektorbibliothek wird auf wichtige Informationen f r die Nutzerfrage durchsucht.

7. **AI-Python-Backend-Server** - Das Ergebnis aus der Vektorbibliothek wird dem Systemprompt + Prompt angehängt.
8. **AI-Python-Backend-Server** → **NodeJS-Server** → **Client** - Das eigentliche Modell wird geladen, die Eingabe wird in Tokens zerlegt und das KI-Modell sagt ein nächstes Token voraus. Dieses Token wird über den NodeJS-Server zum Client gesendet, der es dort visualisiert. Dadurch wird der Eindruck erweckt, die KI, würde live ein Antwort verfassen, was sie ja im Grunde auch tut. Jedenfalls wiederholt sich dieser Vorgang bis das KI-Modell eine <eos>-Token generiert oder der Nutzer die Generierung abbricht.

Warum brauchen wir eigentlich den NodeJS-Server?

Die Frage ist berechtigt, da man ja an sich den NodeJS-Server weglassen könnte und damit ja sichtlich Laufzeit, Energie und Boilerplatte einsparen würde. Aber Python ist **40–70 %** langsamer als NodeJS das Argument mit dem Energie- und Laufzeitsparen ist also aus der Welt und Boilerplatte? NodeJS ist für Server ausgelegt und Javascript für die Webentwicklung. Da beides Javascript ist, ist das Interface fast fließend. Es geht einfach einfacher auf Nodejs einen statischen Server und eine API einzubauen, weshalb der NodeJS-Server doch eine gute Variante ist. Wenn Sie es aber dennoch zu umständlich finden, können Sie den Server gerne umstrukturieren.

4. Trainingsdaten

Das KI-Modell bedient sich aus einer Vektorbibliothek. Der Vorteil bei einer solchen, ist, dass das Modell weniger bis gar nicht mehr haluziniert und die Daten einfacher erneuerbar sind, weshalb die Daten besser *up to date* sind.

4.1 Quellen

Als Quellen für Trainingsdaten können Vorlesungs-Folien, HTWML-Seiten von der offiziellen HTWK-Seite und weitere HTWK-spezifische Texte oder Dokumente wie PDFs oder HTMLs.

4.2 Das Extrahieren von Trainingsdaten

Das KI-Modell benötigt Markdown, da die Ausgabe in Markdown generiert wird und sich Markdown sehr gut für das einfache und schnelle Formatieren von Texten eignet.

Um eine Vektorbibliothek zu bauen ist müssen die Roh-Trainingsdaten in gegliederte Markdown umgewandelt werden. Da meist die Informationsdichte bei z.B. Vorlesungsfolien sehr niedrig ist, da sehr viele unnötige Informationen enthalten sind, viele Informationen auch in Bildern stecken können und kleine lokale KI-Modelle für `nonCUDA`-Rechner nicht geeignet sind, müssen wir auf Cloud-Betriebene Modelle ausweichen. Dafür folgen Sie folgende Schritte für PDF:

1. Laden Sie die gewünschten PDF bei KI-Modellen wie ChatGPT, Gemini oder Claude hoch
2. Geben Sie folgenden Prompt ein:

Du bist ein hochspezialisierter KI-Experte für Datenaufbereitung, Dokumenten-Parsing und Informationsextraktion für RAG-Systeme (Retrieval-Augmented Generation). Deine Aufgabe ist es, die angehängte PDF-Datei (technische Vorlesungsfolien) in eine hochgradig strukturierte, semantische Markdown-Datei zu transformieren. Diese Datei dient als Datengrundlage für eine Vektordatenbank, weshalb die Suchbarkeit und Eigenständigkeit jedes Textabschnitts oberste Priorität haben.

Befolge für das BESTMÖGLICHE Ergebnis diese strikten Anweisungen:

1. Kontext-Injektion (Eigenständigkeit der Chunks garantieren)

Da Textabschnitte in einer Vektordatenbank isoliert indexiert und abgerufen werden, darf kein Chunk interpretationsbedürftig sein.

- Wandle stichpunktartige Folieninhalte in vollständige, grammatikalisch korrekte Fließtexte um.
- Injiziere das übergeordnete Thema (z. B. das Hauptkapitel oder das spezifische Metall) in jeden Sinnabschnitt und jeden Satz, wo es sinnvoll ist. Vermeide isolierte Stichpunkte wie "• Hohe Härte". Schreibe stattdessen: "Die im Prozess der Chrom-Galvanik erzeugte Schicht weist eine sehr hohe Härte auf."

2. Absoluter Fakten- und Datenerhalt (Keine Zusammenfassung)

Erhöhe die Informationsdichte nicht durch Kürzung, sondern durch das Eliminieren von Füllwörtern bei gleichzeitigem Erhalt aller harten Daten.

- Lösche NIEMALS spezifische Fachbegriffe, Kennzahlen, Grenzwerte, Einheiten (z. B. 1000 HV, 200-350 g/L), chemische Verbindungen oder physikalische Parameter.
- Jedes technische Detail muss vollumfänglich im Fließtext erhalten bleiben.

3. Integration handschriftlicher Notizen (Essentiell!)

Das Dokument enthält handschriftliche Ergänzungen und Anmerkungen im Text und am Rand.

- Entziffere alle handschriftlichen Notizen sorgfältig.
- Integriere diese Notizen logisch und semantisch korrekt in den jeweiligen Textabschnitt. Sie enthalten oft wichtige Praxisbeispiele, Ursache-Wirkungs-Prinzipien oder gesetzliche Hintergründe, die auf den gedruckten Folien fehlen.

4. Mathematische und Chemische Formeln (LaTeX-Format)

- Konvertiere sämtliche mathematischen Formeln (z. B. Vickers-Härte, Nernst-Gleichung) sowie alle chemischen Reaktionsgleichungen (z. B. Oxidationsprozesse, Schmelzarbeit) in standardisiertes LaTeX-Format.
- Verwende $\text{\`{}}\text{\`{}}$ für Inline-Formeln im Text und $\text{\`{}}\text{\`{}}\text{\`{}}$ für freistehende, zentrierte Formeln.

5. Verbalisierung von Diagrammen und technischen Zeichnungen

Lösche keine Bilder, die technischen Inhalt transportieren.

- Analysiere Grafiken wie Tabellen, Schichtaufbauten (z. B. Tiefdruckzylinder-Schicht en), Schaltbilder (z. B. galvanische Zellen) oder Diagramme (z. B. E/pH-Diagramme).
- Übersetze die visuelle Information, Beschriftungen, Reihenfolgen und Prozesse vollständig in präzisen beschreibenden Text oder übersichtliche Markdown-Tabellen.

6. Bereinigung von Rauschen (Globale Meta-Informationen entfernen)

Entferne alle Elemente, die für eine semantische Inhaltssuche wertlos sind oder Chunks verfälschen:

- Keine Titelfolien, Inhaltsverzeichnisse oder Kapitelübersichten.
- Keine Seitenzahlen, Logos, wiederkehrende Kopf- oder Fußzeilen (z. B. Universitätsnamen, Modulnummern, Autoren).
- Keine reinen Schmuckbilder (z. B. historische Gemälde).
- Keine Kontaktdaten oder Abschlussfolien.

7. Striktes Ausgabeformat (Code-Block)

- Umschließe den gesamten generierten Markdown-Inhalt zwingend mit einem einzigen, zusammenhängenden Markdown-Code-Block (unter Verwendung von dreifachen Backticks und dem Sprachbezeichner `markdown`, also: ````markdown [Inhalt] ````). Das ist essenziell, damit der Endnutzer das Ergebnis sauber herauskopieren kann.
- Nutze innerhalb dieses Blocks eine klare, flache Hierarchie mit `##` und `###` für semantische Abschnitte.
- Gib KEINERLEI Einleitungstext, Metatext, Höflichkeitsfloskeln oder Schlusssätze außerhalb oder innerhalb des Code-Blocks aus (z. B. KEIN "Hier ist dein konvertiertes Markdown:"). Die gesamte Antwort darf ausschließlich aus diesem einen Code-Block bestehen.

1. Nutzen Sie idealerweise den `Think-Mode` oder `Pro-Mode` (bei Gemini und Claude) und schicken Sie ab.

2. Kopieren Sie den erzeugten Output in eine Datei mit dem Namen `name_ihrer_input_datei_ohne_endung.md`.

Falls Sie eine HTML haben befolgen Sie folgenden Prompt:

Du bist ein hochspezialisierter KI-Experte für Web-Parsing, HTML-Strukturierung und Informationsextraktion für RAG-Systeme (Retrieval-Augmented Generation). Deine Aufgabe ist es, den bereitgestellten HTML-Quelltext in eine hochgradig saubere, semantische und dichte Markdown-Datei zu transformieren. Diese Datei dient als Datengrundlage für eine Vektordatenbank, weshalb die Eigenständigkeit jedes Textabschnitts oberste Priorität hat.

Befolge für das BESTMÖGLICHE Ergebnis diese strikten Anweisungen:

1. Radikale Bereinigung von Rauschen (Boilerplate-Removal)

Entferne vor der Konvertierung alle HTML-Elemente, die keinen inhaltlichen Mehrwert für eine Wissensdatenbank bieten:

- Lösche jegliche Navigationselemente (<nav>), Footer (<footer>), Header/Kopfzeilen (<header>), Sidebars (<aside>).
- Entferne Cookie-Banner, Werbe-Anzeigen, Social-Media-Buttons, Kommentarspalten und rechtliche Hinweise (Impressum, Datenschutz).
- Ignoriere alle Skripte (<script>), Styles (<style>) und Meta-Tags, die nicht den Kerninhalt betreffen.
- Konzentriere dich ausschließlich auf den Hauptinhalt (z. B. innerhalb von <main>, <article> oder den zentralen Inhalts-<div>).

2. Kontext-Injektion & Chunk-Eigenständigkeit

Da das Markdown später in isolierte Abschnitte (Chunks) zerteilt wird, darf kein Abschnitt interpretationsbedürftig sein.

- Wenn das HTML tief verschachtelt ist (z. B. H1 -> H2 -> H3 -> H4), injiziere den Kontext der übergeordneten Überschriften (z. B. das Hauptthema) geschickt in die Unterüberschriften oder direkt in die Absätze.
- Vermeide isolierte Sätze oder unvollständige Fragmente. Wandle kurze, unverständliche Phrasen in grammatikalisch vollständige Fließtexte um, die das Thema explizit nennen.

3. Konvertierung von Tabellen und Listen

- Wandle HTML-Tabellen (<table>) in saubere Markdown-Tabellen um. Wenn eine Tabelle zu komplex ist oder Gefahr läuft, beim Chunking zerschnitten zu werden, verbalisiere die Tabellenzeilen stattdessen in informationsreiche Sätze (z. B. "Das Attribut X hat den Wert Y").
- Wandle ungeordnete () und geordnete Listen () in Standard-Markdown-Listen um, sofern sie im Kontext verständlich bleiben.

4. Erhalt von technischen Daten und Quellcode

- Lösche NIEMALS Kennzahlen, Einheiten, Grenzwerte oder technische Fachbegriffe.
- Wenn das HTML Programmiercode enthält (<pre><code>), übernehme diesen exakt in Mark

down-Code-Blöcke und füge den passenden Sprachbezeichner (z. B. ````python`) hinzu.

5. Umgang mit Bildern (Alt-Texte nutzen)

- Lösche keine Bilder, die Information tragen. Nutze die Attribute ``alt`` und ``title`` von `<img>`-Tags.
- Übersetze den Inhalt des ``alt``-Attributs sowie eventuelle Bildunterschriften (`<figcaption>`) in eine präzise Textbeschreibung im Markdown, damit der Bildinhalt textlich durchsuchbar wird.

6. Mathematische und Chemische Formeln

- Falls das HTML Formeln (z. B. via MathJax, MathML oder als Text) enthält, konvertiere diese in standardisiertes LaTeX-Format. Nutze ``$`` für Inline-Formeln und ``$$`` für freistehende Blöcke.

7. Striktes Ausgabeformat (Code-Block)

- Umschließe den gesamten generierten Markdown-Inhalt zwingend mit einem einzigen, zusammenhängenden Markdown-Code-Block (unter Verwendung von dreifachen Backticks und dem Sprachbezeichner ``markdown``, also: ````markdown [Inhalt] ````). Das ist essenziell für den sauberen Export.
- Nutze eine flache, klare Überschriftenhierarchie (bevorzugt ``##`` und ``###``).
- Gib KEINERLEI Einleitungstext, Metatext oder Schlusssätze außerhalb oder innerhalb des Code-Blocks aus. Die Antwort muss ausschließlich aus diesem einen Code-Block bestehen.

Falls Sie aber schon eine Markdown haben nutzen sie folgenden Prompt:

Du bist ein hochspezialisierter KI-Experte für Datenaufbereitung, Text-Refactoring und Informationsextraktion für RAG-Systeme (Retrieval-Augmented Generation). Deine Aufgabe ist es, die bereitgestellte, unstrukturierte Markdown-Datei (die aus einer PDF-Konvertierung von Vorlesungsfolien stammt) komplett zu überarbeiten und für eine Vektordatenbank zu optimieren.

Befolge für das BESTMÖGLICHE Ergebnis diese strikten Anweisungen:

1. Kontext-Injektion (Auflösung von Stichpunkt-Fragmenten)

Da die Datei später in isolierte Chunks zerteilt wird, darf kein Abschnitt interpretationsbedürftig sein.

- Wandle abgehackte Stichpunkte, unvollständige Sätze und lose Textfragmente in grammatikalisch korrekte, informationsdichte Fließtexte um.
- Injiziere das übergeordnete Thema (z. B. das Hauptkapitel wie "Chrom-Galvanik" oder "Nickel-Galvanik") aktiv in jeden Sinnabschnitt und jeden Satz, wo es für das Verständnis nötig ist. Vermeide isolierte Aussagen wie "• Hohe Härte". Schreibe: "Die durch Chrom-Galvanik erzeugte Schicht besitzt eine hohe Härte."

2. Logische Integration handschriftlicher Notizen

In der bestehenden Markdown-Datei sind handschriftliche Notizen oft ungeordnet am Ende einer Seite oder mitten im Text als Fragmente eingestreut (erkennbar an Formulierungen wie "Notiz:", "Anmerkung:" oder losen Stichpunkten wie "Verbot Chrom G", "viel Gas").

- Analysiere, zu welchem fachlichen Thema diese Notizen gehören.
- Integriere die handschriftlichen Zusatzinformationen logisch, flüssig und semantisch korrekt in den Fließtext des passenden Abschnitts, da sie oft wichtige Kausalitäten oder Praxisbeispiele enthalten.

3. Absoluter Fakten- und Datenerhalt (Keine Zusammenfassung)

- Kürze den Text nicht im Sinne einer Zusammenfassung. Erhöhe die Informationsdichte ausschließlich durch das Streichen von Füllwörtern.
- Alle technischen Daten, Kennzahlen, Grenzwerte, Einheiten (z. B. 1000 HV, 200-350 g/L) und chemischen Bezeichnungen müssen zwingend und vollständig im Fließtext erhalten bleiben.

4. Mathematische und Chemische Formeln (LaTeX-Format)

- Überprüfe alle mathematischen Formeln und chemischen Reaktionsgleichungen.
- Konvertiere sie konsequent in standardisiertes LaTeX-Format. Verwende $\dots$ für Inline-Formeln und $\dots$ für freistehende Formelblöcke.

5. Bereinigung von Rauschen (Metadaten entfernen)

- Entferne alle Fragmente, die von Folien-Kopf- und Fußzeilen stammen (z. B. Seitenza

hlen, Modulnamen wie "I755 Materialwissenschaften", Hochschulnamen wie "HTWK Leipzig", Autorennamen, "Übersicht"-Folien).

- Bereinige leere oder zerschossene Bild-Tags (z. B. ``) , es sei denn, der Alt-Text enthält verwertbare technische Beschreibungen - falls ja, verbalisiere diese Beschreibung in echten Text.

6. Striktes Ausgabeformat (Code-Block)

- Umschließe den gesamten neu strukturierten Markdown-Inhalt zwingend mit einem einzigen, zusammenhängenden Markdown-Code-Block (unter Verwendung von dreifachen Backticks und dem Sprachbezeichner `markdown`, also: ```markdown [Inhalt] ```). Das ist essenziell für den sauberen Export.

- Nutze eine klare, flache Hierarchie mit `##` und `###` für die semantischen Abschnitte.

- Gib KEINERLEI Einleitungstext, Metatext oder Schlusssätze außerhalb oder innerhalb des Code-Blocks aus. Die gesamte Antwort darf ausschließlich aus diesem einen Code-Block bestehen.

4.3 Das Bauen einer Vektorbibliothek

Nehmen Sie zunächst die erzeugte Markdown und verschieben Sie diese in den Ordner `./scientia/data/veclib/raw`, was Sie auch ganz einfach mit folgenden Befehl tun können:

```
./sce.bat lib add ihre_md_datei.md
```

Falls Sie kein Administrator sind bzw. keinen Zugriff auf dem Server haben, schicken Sie die erzeugte Datei zu den Administrator des Servers.

Wenn Sie Administrator sind: Wenn Sie alle Markdowns gesammelt haben, können Sie eine Vektobibliothek bauen. Dies tun sie mit folgenden Befehl:

```
./sce.bat lib build
```

Als Ergebniss bekommen Sie eine Vektorbibliothek unter `./data/veclib/vektorbases.index` und `./data/veclib/text_chunks.pkl`.

5. Befehle

Alle Befehle funktionieren unter Windows und Linux.

- Alles in `[]` ist optional.
- Alles in `<>` ist ein Platzhalter und muss ersetzt werden.
- `|` bedeutet "oder".

Hilfe anzeigen

```
./sce.bat --help
```

Zeigt die vollständige Befehlsübersicht an.

NodeJS-Server starten

```
./server.bat
```

Startet den NodeJS-Server.

Python-Backend starten

```
./sce.bat server
```

Startet das AI-Python-Backend.

Modell finetunen

```
./sce.bat train finetune \  
    [-i <inputmodell>] \  
    [-o <outputmodell>] \  
    [-v <vektormodell>] \  
    [-d <trainingsdaten>] \  
    [-l <vektorbibliothek>]
```

Trainiert bzw. finetuned ein vorhandenes Sprachmodell.

Trainingsdaten aufteilen

```
./sce.bat train split \  
    [-f <all.jsonl>] \  
    [-d <ausgabeordner>]
```

Teilt `all.jsonl` automatisch in `train.json` und `val.json` auf.

Markdown-Datei zur Vektorbibliothek hinzufügen

```
./sce.bat lib add <datei> [-r]
```

Fügt eine Markdown-Datei hinzu.

Mit `-r` wird sie stattdessen im RAW-Ordner (`data/veclib/raw`) gespeichert.

Trainingsdaten auflisten

```
./sce.bat lib list [-r] [-i <glob> ...]
```

Listet alle Trainingsdaten auf.

Optionen:

- `-r` zeigt den RAW-Ordner an.
 - `-i` ignoriert Dateien oder Ordner anhand von Glob-Patterns.
-

Trainingsdaten entfernen

```
./sce.bat lib remove <datei> [-r]
```

Entfernt eine Datei aus der Vektorbibliothek.

Mit `-r` wird aus dem RAW-Ordner gelöscht.

Vektorbibliothek erzeugen

```
./sce.bat lib build \  
    [-s <markdownordner>] \  
    [-d <ausgabeordner>]
```

Erstellt aus den Markdown-Dateien eine neue Vektorbibliothek.

PDFs nach Markdown konvertieren

```
./sce.bat lib convert \  
    [-s <pdf-ordner>] \  
    [-d <markdown-ordner>]
```

Konvertiert PDFs in Markdown und speichert sie im Zielordner.

KI-Modell herunterladen

```
./sce.bat download \  
    [-m <huggingface-modell>] \  
    [-p <zielordner>]
```

Lädt ein Modell von Hugging Face herunter.

Werden keine Parameter angegeben, werden die Standardmodelle heruntergeladen.

Modell in der Konsole testen

```
./sce.bat test
```

Startet den Chatbot direkt in der Konsole.

6. Starten (Unter UNIX: Linux/macOS von 0)

Falls Sie Windows nutzen, schauen Sie oben oder lassen Sie sich alle UNIX von einer KI in eine Anleitung für Windows umwandeln.

1. NodeJS installieren

Das tun Sie mit folgenden Befehl:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/master/install.sh | bash && source  
  ~/.bashrc && nvm install 24.11.0 && nvm use 24.11.0
```

Dannach:

```
node -v | npm -v
```

Wenn dann folgendes kommt war die Installation erfolgreich:

```
v24.11.0  
v11.6.1
```

1. Python installieren

Installieren Sie Python mit den folgenden Befehlen:

Ubuntu/Debian: Build-Abhängigkeiten installieren

```
sudo apt update  
sudo apt install -y \  
build-essential \  
curl \  
git \  
libssl-dev \  
zlib1g-dev \  
libbz2-dev \  
libreadline-dev \  
libsqlite3-dev \  
libffi-dev \  
libncursesw5-dev \  
libgdbm-dev \  
libnss3-dev \  
liblzma-dev \  
tk-dev \  
uuid-dev \  
libxml2-dev \  
libxmlsec1-dev \  
xz-utils
```

Python installieren

```
curl https://pyenv.run | bash
```

```
echo 'export PYENV_ROOT="$HOME/.pyenv"' >> ~/.bashrc
echo '[[ -d $PYENV_ROOT/bin ]] && export PATH="$PYENV_ROOT/bin:$PATH"' >> ~/.bashrc
echo 'eval "$(pyenv init - bash)"' >> ~/.bashrc

source ~/.bashrc

pyenv install 3.13.14
pyenv global 3.13.14
```

Überprüfen Sie anschließend die Installation:

```
python --version
```

Bekommen Sie folgendes als Output waren Sie erfolgreich. Glückwunsch!:

```
#Python 3.13.14
```

1. Scientia-Core herunterladen

Zuvor müssen Sie `git` installiert haben. Geben Sie folgendes ein:

```
sudo apt install git
```

Geben Sie einmal folgendes ein wenn eine Version erscheint haben Sie git erfolgreich installiert:

```
git --version
```

Jetzt bewegen Sie sich mit `cd <relativer_pfad>` (`..` heißt einmal herausnavigieren) zu ihren Ordner, wo der Chatbot liegen soll und laden Sie Scientia-Core mit folgenden Command herunter:

```
git clone https://github.com/Pydalo/scientia-htwk
```

Nun liegt im Unterordner `/scientia-htwk` Scientia-Core. Navigieren Sie auch einmal in den Ordner herein:

```
cd ./scientia-htwk
```

1. NodeJS-Abhängigkeiten installieren

Navigieren Sie in den Unterordner `./server` und installieren Sie alle NodeJS-Abhängigkeiten mit folgenden Command:

```
npm install
```

1. Python-Abhängigkeiten installieren

Navigieren Sie mit `cd ..` aus dem Ordner heraus und navigieren Sie dann in den Ordner `/scientia`. Geben Sie dann folgendes ein, um ein Virtual-Environment zu erstellen:

```
python -m venv ./venv
```

Gefolgt vom Folgenden um alle Python-Abhängigkeiten zu installieren (*Kann je nach Internetconnection ne weile dauern. Hat man Zeit sich nen Kaffee zu machen*):

```
./venv/bin/pip install -r requirements.txt
```

1. KI herunterladen

Navigieren Sie aus dem jetzigen Ordner heraus. Auf Linux ist das Problem, dass Sie beim aufrufen der Datei `./sce.bat` einen Fehler bekommen, da der Datei Berechtigungen fehlen. Weil Sie aber nicht immer `sudo bash ./sce.bat ...` benutzen wollen geben Sie einmal ein:

```
chmod +x sce.bat  
chmod +x server.bat
```

und geben Sie ein:

```
./sce.bat download
```

